AI 407 Introduction to Robotics
Final Project Report

TurtleBot3 Autonomous Object Retrieval and Placement in a Maze Environment with ROS

Team Members: [Name 1], [Name 2], [Name 3], [Name 4]
Lab Section: [Section] Instructor: [Instructor]
Date: [Submission Date]

1. Project Description

The project implements a simulation of a TurtleBot3-style robot equipped with a simple manipulator representation. The robot operates inside a 7x7 maze, starts from cell (1,3), searches for two objects located in the same unknown cell, reports the discovered object cell, transports both objects one at a time to a target cell, and finally returns to the original starting point.

2. Maze and Location Metrics

The maze follows the required coordinate convention, where cells are named by row and column from the bottom starting side. The simulation uses the following metric scale:

Requirement	Simulation Value
Maze size	3.5 m x 3.5 m (350 cm x 350 cm)
Grid	7 x 7 cells
Cell size	0.5 m x 0.5 m (50 cm x 50 cm)
Wall height	0.20 m (20 cm)
Wall thickness	0.015 m (1.5 cm)
Start cell	(1,3)
Target cell	(1,7)
Object demo cell	(5,4)
Object dimensions	0.02 m x 0.03 m x 0.20 m

The Gazebo world uses separate wall models so the maze visually represents stand-alone MDF wall segments rather than one single attached mesh.

3. System Architecture

- Gazebo world: contains the 7x7 maze, start marker, target marker, two objects, and a simple TurtleBot3-with-arm visual model.
- ROS 2 package: contains the launch file, world file, map description, and autonomous mission node.
- Maze task node: controls the mission sequence, stores discovered paths, and moves the simulated robot and objects during the demonstration.
- Path memory: stores the path to the object cell, the path to the target cell, and the return path to the starting cell.

4. Implemented Algorithm Overview

The algorithm combines cell-based maze navigation, search behavior, path memory, object retrieval, and target delivery. The robot initially explores the maze using a trial-and-error style search sequence. When the object cell is reached, the system displays the required discovery message: Found (5,4). After discovery, the robot stores useful paths and reuses them for efficient navigation instead of repeating the initial search.

- Initialize the robot at cell (1,3).
- Search cell-by-cell until the objects are discovered.
- Print the object location as Found (5,4).
- Store the path from start to object cell.
- Use BFS-style path planning between object cell and target cell.

- Pick object 1, move to the target cell, and release it.
- Return to the object cell using the remembered route.
- Pick object 2, move to the target cell, and release it.
- Return to the original start cell and stop.

5. Simulation Execution

The simulation is launched from a ROS 2 workspace after building the package with colcon. The launch file opens the Gazebo world and starts the autonomous task node after Gazebo finishes loading.

Commands used:

```
cd ~/ai407_ws

colcon build

source install/setup.bash

ros2 launch ai407_maze_project simulation.launch.py
```

6. Object Detection and Manipulator Behavior

In this simulation version, the two objects are placed in the same cell with the required dimensions. The task node demonstrates the discovery event and object handling behavior. The manipulator action is represented by attaching the object visually near the robot during transport and releasing it inside the target cell. This keeps the project focused on the required mission logic while remaining executable in a Gazebo simulation environment.

7. Testing and Results

Test	Expected Result	Status
Gazebo world loads	7x7 maze appears with walls, start, target, objects, and robot model	Completed
Start coordinate	Robot starts from cell (1,3)	Completed
Object discovery	Terminal displays Found (5,4)	Completed
First delivery	Object 1 is transported to target cell (1,7)	Completed
Second delivery	Robot returns to object cell and delivers object 2	Completed
Return home	Robot returns to cell (1,3) and stops	Completed

8. Teamwork and Role Distribution

The project was divided into clear roles so the team could work in parallel while still integrating the final simulation together.

Team Member	Main Responsibility
Member 1	Gazebo maze design and world file testing
Member 2	ROS 2 package setup, launch file, and simulation execution
Member 3	Navigation algorithm, path memory, and mission sequence
Member 4	Report, presentation, video explanation, and final demo preparation

9. Challenges in Teamwork

- Keeping the same Ubuntu 22.04 and ROS 2 Humble environment across devices was necessary to avoid package mismatch errors.
- The maze dimensions had to be checked carefully because Gazebo uses meters while the project specification uses centimeters.
- The team had to separate Phase 2 planning from Phase 3 implementation to avoid overcomplicating the proposal stage.
- Integration required communication between the maze design, ROS node, and presentation/report parts.

10. Conclusion

The final simulation demonstrates the required autonomous object retrieval and placement workflow in a 7x7 Gazebo maze. The robot starts from the given coordinate, discovers the object cell, stores and reuses paths, delivers both objects to the target cell, and returns to its original parking point. The package is prepared for simulation submission and final demo presentation.

References

1. ROS 2 Humble Documentation.
2. Gazebo Classic Documentation.
3. TurtleBot3 e-Manual, ROBOTIS.
4. OpenManipulator-X e-Manual, ROBOTIS.
5. AI 407 Project Guidelines and Laboratory Manuals.